Chai aur React (Part 3)

— Taani

1

Why should we learn react?

⇒ shipping is fast, bcs of component-based architecture

⇒ built on top of JS

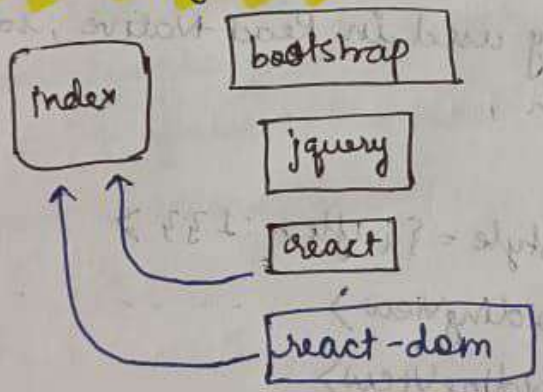
⇒ React is a library, not more

Frontend has two parts:

CSR (Client Side Rendering)

SSR (Server Side Rendering)

* SSG (Static site generation)



'unpkg' lets us use npm packages directly in the browser without installing them.

Nodes in Virtual DOM are called fibres.

Create Element in React

```
React.createElement(
    type,
    props,
    children
)
```

inside 'App.js'


```
const App = () => {
    return React.createElement(
  "div",
    {
    className: "container",
  },
  React.createElement("h1", null, "Hello, React!"),
);
};
const container = document.getElementById("root");
const root = ReactDOM.createRoot(container);
root.render(React.createElement(App));
```

React DOM gives us method "create Root"

Components in React

```
const Chai = () => {
    return React.createElement(
  "div",
  {},
  [
  React.createElement("h1", null, "Hello, chai!"),
  React.createElement("p", null, "This is a react
                                  component."),
  ]);
};
```

Then add this Chai component to the main App component.

```
React.createElement(Chai, ~~null~~, {
    name: "Masala Chai",
    desc: "This is authentic Chai"
},
```

} prop (also known as properties)

The components which do not have props are called 'Dumb components';

~~Est~~ Inter help you format, fix messy code, also checks rules/issues. eg ESLint, Biome, JSLint etc

- ESLint can catch bad practices, potential bugs, wrong React hooks usage, missing dependencies etc.

Prettier is just for formatting. Your project will have 'prettier' file.

ESLint has '.eslintignore', and prettier has 'prettierignore' also ESLint has a configuration file 'eslint.config.mjs'

Good practice to use both ESLint + Prettier in project.

JSX ⇒ JS + HTML

~~Library to write JSX (Babel)~~. Babel is a JS compiler. It converts the JSX codes to old JS code which browser can understand

~~Ex~~ JSX → Babel compiles JS toolchain → React

your modern code → Babel → Browser-compatible JS.

To use it, <script type="text/babel">

To create React component manually using React.create Element() and props.


```
const chai = (props) => {
    return React.createElement("div", {}, [
React.createElement("h1", null, props.name || "Chai"),
React.createElement("p", null, props.desc || "demo
component"));
]);
};
```

↑ If desc exists use it, if not use 'Chai component'.

→ Props is like the data passed into component.

To use vite's React plugin, use cmd:

npm install --save-dev vite @vite/plugin-react

Even after this you will run into errors regarding JSX, so, make changes in 'vite.config.js' file.

```
import {defineConfig} from "vite";
import react from "@vitejs/plugin-react";
export default defineConfig({
    plugins: [react()],
});
```

→ Keep all files extension as '.jsx'.

In Vite, .env files are used to store environment variables like:

→ API URLS

→ secret keys etc.

In VITE, variables starting with VITE_, to expose to frontend code.

Note. .env in frontend is NOT secure

CORS → This issue happens when frontend and backend are running on different origins., so browser blocks requests for security.

CORS is a rule which says 'Backend must explicitly allow frontend requests, meaning 'which frontend can access which backend.

Proxy : A proxy acts like a middleware, Instead of Frontend directly accessing backend, we do.
Frontend → Vite server → Backend to avoid CORS

In useEffect(() => {
```
fetch(`${import.meta.env.VITE_API_URL/all-chai`)
},[])
```
↑ give empty array or it will become an infinite loop.

Vite config will have :
```
export default defineConfig({
    plugins : [react()],
    server : {
    proxy : {
        "/api" : {http://localhost : 3000",
        changeOrigin : true,
    },
},
});
```

⇒ The config file code tells Vite,
→ if request starts with /api } to avoid CORS problems
send it to localhost:3000

Always build project before deploying.

⇒ ~~const~~ –

hook fun

```
const [data, setData] = useState(null);
```

data ⇓ variable
setData ⇓ fun
null ↑ initial state

Good vs Bad Practice

```
setCount(count+1)
setCount(count+1)
```
✗

Better,
```
setCount(prev => prev+1)
setCount(prev => prev+1)
```
✓

Bcs the data is async, so can be batched, so in first practice code, React gives same output for both times sometimes

Use Effect

Syntax:
```
useEffect() => {
  // effect logic
  return () => {
    // cleanup logic
  }
}, [dependencies])
```

In React, "key" is a special prop used while rendering lists.

7

* if a key is same, React can reuse the DOM NODE
* if a key is new, react adds a new DOM node
* if a key is missing, react remove the node

* If key is missing, Incorrect re-ordering can happen, animation, can cause UI bugs.

Custom Hook

```
import {useState, useEffect} from 'react'
export function useSpecialChai() {
    const [chai, setChai] = useState(null);
    const [loading, setLoading] = useState(true);
    const [error, setError] = useState(null);

useEffect(() => {
    fetch(`${import.meta.env.VITE_API_URL}/all-chai`)
    .then(response => {
        if (!response.ok) {
            throw new Error('Network response was
                not ok');
        }
        return response.json();
    })
    .then(data => {
        ~~setD~~ setChai(data)
        setLoading(false);
    })
```

```
• catch ((error) => {
        setError (error);
        setLoading (false);
  }, []);
  return {chat, loading, error};
}
```

SSG (Static Site Generation)
means HTML pages are generated at build time, before user visits the website.

Build time
↓
Generate HTML files
↓
Deploy
↓
Users get fast static pages.

Gatsby is a React based framework mainly used for building static websites.

d build.js

```
import fs from "fs-extra";
import path from "path";
import {fileURLToPath} from "url";
import App from "../src/App.js";
import React from "react";
//GET _ dirname
const outputPath = path.join (_dirname, "../dist");
      htmlTemplatePath = path.join (_dirname, "./src/template.html")
      outputHTMLPath = path.join (outputPath, "index.html");
```

```
const template = fs.readFileSync(htmlTemplate, "utf-8");
```

```
const appHtml = ReactDOMServer.renderToStaticMarkup(React.createElement(App, {teas}));
```

↑ This is in memory

→ This code is doing server-side HTML generation using ReactDOMServer.

'.renderToStaticMarkup' ⇒ converts component into static HTML.

→ But we need it in template so

```
const = teamplate.replace("<!--app-->", appHtml);
```

final HTML

```
// write output HTML file
fs.ensureDirSync(outputPath);
fs.writeFileSync(outputHtmlPath, finalHTML, "utf-8");
console.log("Build completed Output written to dist/index.html");
```

Next JS from scratch

Run cmd: npm i react react-dom express

Code: App.js

```
import React from 'react';
export default fn" App() {
    return React.CreateElement(
```

```
"div",
{style: {color: "orange"}},
React.createElement("h1", null, "Hello, world!"),
);
}
```

10

Now in server.js

```
import React from 'react'
import express from express;
import ReactDomServer from "react-dom/server";
import App from "./src/App.js";
const app = express();
app.get("/", (req, res) => {
    res.send("Hello world");
});
app.get("/chaicode", (req, res) => {
    const appHtml = ReactDOMServer.renderToString(
        React.createElement(App));
    res.setHeader("Content-Type", "text/html");
    res.send(`
      <!DOCTYPE html>
      <html lang="en">
      <head>
          <meta charset="UTF-8" />
      </head>
      <body>
        <div id="root">
```

SSR

- SSG (Static Site Generation) eg Gatsby
- ISG (Increment Site Generation), only re-builds the files changed (only generate this) 11
- Dirty files (the files which have been changed)

SSR will always have latest data, SSG has done API call once, and kept data somewhere once.

For interviews,

```
fun App(){
  const [count, setCount] = useState(0);
return (
  <>
  <h1> Chai aur date night react </h1>
  </>
);
}
export default App;
```

- In "BrokenCup.jsx"

```
import {useState} from "react";
export fun BrokenCup(){
  const [isBroken, setIsBroken] = useState(false)
if (isBroken){
      throw new Error ("The cup is broken!");
}
return (
```

```
<div>
    <p> The cup is intact. </p>
<button onClick={() => setIsBroken(true)}>Break the cup</button>
</div>
```

Error Boundaries : The components that catch JS errors and show fallback UI, instead of crashing the whole App

Enclose the component which can potentially crash or give errors inside the <Error Boundary> component.

→ without Error boundaries, one broken component destroys /crashes entire app, without giving errors.

Rather than writing custom, use cmd :

⇒ npm install react-error boundary